TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Context-File Redis and High-Performance Storage

An HPE Juniper Networking Techpost

Christian Graf

2026-05-14

Contents

1 Context-File Redis and High-Performance Storage 2
1.1 Introduction . . . 2
1.2 Why Persistent Storage? . . . 3
1.3 Why Redis? . . . 3
1.4 Performance Comparisons . . . 4
1.5 Performance Summary . . . 4
1.6 Installing Redis . . . 5
1.7 Ubuntu/Debian . . . 5
1.8 macOS . . . 5
1.9 Starting Redis . . . 5
1.9.1 Option A: Background Service (Recommended) . . . 5
1.9.2 Option B: Foreground Process . . . 5
1.10 Verify Redis is Running . . . 5
1.11 Context File Redis Prerequisites . . . 6
1.12 Quick-Start: End-to-End Redis Example . . . 6
1.12.1 Phase 1: Read Back . . . 7
1.13 Redis Storage: Master Index and TTL . . . 8
1.14 Writing Data: setex and hset Together . . . 9
1.15 Why TTL Matters . . . 10
1.16 Querying: Reading from the Master Index . . . 10
1.17 Verifying TTL in shell . . . 10
1.18 Context File Master Index Usage . . . 11
1.19 Redis TTL and Phase 3 Trending . . . 14
1.20 TTL as the Look-Back Mechanism . . . 14
1.21 Configuring N and TTL Together . . . 14
1.22 Tracking Phase 0 Runs for Phase 3 . . . 15
1.23 Handling Expired Runs in Phase 3 . . . 16
1.24 Summary . . . 16
1.25 Next in the Series . . . 17
1.26 Glossary . . . 17

1 Context-File Redis and High-Performance Storage

Christian Graf - 05/15/2026

Fifth part of the Series on the Context-Files and JMCP, covering Redis and high-performance Storage.

Series Navigation:

- Part 1: Introduction & Basics
- Part 2: Tokens & Memory
- Part 3: Optimization & Alerting
- Part 4: File Management
- Part 5: Redis & Performance <- You are here
- Part 6: Hybrid Architecture

1.1 Introduction

In Parts 1–4, we covered context file fundamentals, token management, optimization & alerting, and file-based data collection. In this one, we'll explore Redis as a high-performance caching layer that enables massive scale and sub-second data access.

This blog uses a four-phase workflow model introduced in Part 4. As a quick reference:

Phase	Name	Scope
0	Collection	Query devices via JMCP, parse output, store results in Redis or files
1	Analysis	Load cached data, check freshness, auto-recover stale entries, run domain checks (e.g. LSDB sync)
2	Correlation	Cross-domain causal analysis across all Phase 1 domain reports
3	Trending	Compare N recent snapshots over time to detect drift and recurring failures

Each phase can run in a separate LLM session. Redis (or files) carries data between them.

> **Note for enhanced JMCP users**: If you are using the enhanced JMCP server in artifact mode (introduced in Part 4), Redis is already your storage backend - the server writes the data collection in Phase 0 output directly to Redis or files without touching the LLM context window. This blog covers the key/TTL patterns, Master Index design, Phase 3 trending with TTL-controlled look-back windows, external Python analysis, and the Redis vs Files decision matrix that apply to both storage backends.

1.2 Why Persistent Storage?

LLM sessions are stateless. When a chat ends, everything in the context window - all the raw CLI output, all the parsed results, all the intermediate analysis - is gone. The next session starts with a blank slate.

For one-shot ad-hoc investigations this is fine. But for any workflow that repeats - daily health checks, hourly trending, multi-phase pipelines that span several LLM sessions - you need somewhere outside the LLM to persist data between runs. Part 4 covered file-based persistence extensively. This part introduces Redis as a complementary storage backend and explains when it outperforms files:

Files - plain JSON or text written to disk. No infrastructure required, human-readable, easy to audit. Part 4 covered this extensively: Phase 0 writes cache files, and subsequent analysis phases load them. For most teams, files are the right starting point.

Redis - an in-memory key-value store. Requires a running Redis service, but delivers sub-millisecond access, native TTL-based automatic expiry, and structured query patterns that scale to very large fleets. When the workflow runs regularly and the dataset grows large, Redis starts to outperform files.

The rest of this part explains when to choose Redis, how to configure it, and how its TTL mechanism directly controls the Phase 3 (Trending) look-back window. The Redis vs Files comparison below provides a side-by-side summary.

1.3 Why Redis?

Redis is an in-memory key-value store. Because all data lives in RAM, read and write operations are sub-millisecond - far faster than disk-based file I/O. Keys carry a built-in TTL, so old runs expire automatically without manual cleanup scripts. Its key-value and HASH structures support targeted queries - fetch the latest snapshot for one router without scanning everything - which keeps access patterns efficient as the fleet grows. And when paired with external Python scripts that analyse data entirely outside the LLM, token cost stays fixed regardless of fleet size: the context window never sees raw per-router data at all.

Side-by-Side Comparison:

Factor	Redis	Files
Speed	Fastest (memory)	Fast (disk I/O)
Scale	Excellent	Good
TTL/Auto-expiry	Yes (automatic)	No (manual cleanup)
Audit Trail	No (data expires)	Yes (permanent)
Setup Complexity	Medium (Redis service)	Low (filesystem)
Token Efficiency	Excellent (external scripts)	Excellent (external scripts)
Human Readable	No (binary)	Yes (JSON/text)
Infrastructure	Requires Redis server	None (just filesystem)
Best For	Production, trending	Development, audits

1.4 Performance Comparisons

Redis alone provides no performance advantage - the gain comes from pairing it with external Python analysis. The following scenarios show when and how that combination unlocks significant scale and performance benefits. When working with an MCP server, multiple scenarios exist for obtaining data to analyze. The most obvious is to use JMCP: call any tool to gather data and all tool output arrives directly in the context window. Based on the task, the LLM directly parses and analyzes the data - or, if something more complex is required, it generates Python code on the fly to solve it. This is very convenient; the LLM appears capable of solving almost anything, especially during ad-hoc troubleshooting. A clear limit is performance and scale: analyzing large datasets such as **show isis database extensive** from many routers will likely exhaust the context window. The LLM works around this by batching - loading only partial data into context and solving the task sequentially. The biggest disadvantage of batching is that data is queried in multiple steps with visible delays: the **show isis database** snapshot from the first router may be taken minutes apart from the last router's LSDB snapshot, making any cross-router comparison temporally inconsistent.

Another solution which gets covered in more detail in this blog, is to query all data in a single shot and save it to files or a Redis memory cache. The biggest advantage of this approach is that all data comes from the same snapshot. However, with respect to the context window, it is not fundamentally different from the first approach - the data still needs to be loaded into context for the LLM to analyze it.

The most performant solution, described in depth in the upcoming Part 6 of this series, is the combination of Redis - to collect all data in a single synchronized snapshot - and external Python for scaled, high-performance analysis entirely outside the context window.

Using databases for storage is out of scope for this blog. Databases would serve equally well as a persistent storage solution.

1.5 Performance Summary

JMCP Direct works well for ad-hoc investigations and smaller fleets, but operational health checks that cover many domains or large fleets quickly push context window limits - the context file either grows very large or must be split across several domain-specific files. Inline Python, as used in this blog, keeps the context file and Python logic together in a single place, which is convenient as a learning example but is not suited for production: token cost scales with fleet size, and analysis logic embedded in a context file is hard to version and maintain. The real goal for daily routine operations - recurring health checks, trending, cross-domain failure correlation - is to use the LLM for what it excels at: reasoning across heterogeneous signals and surfacing causal chains that span multiple failure domains. The raw data processing and comparison work is delegated to external Python scripts that run entirely outside the context window, keeping token cost fixed regardless of fleet size. The trade-off is a meaningfully higher maintenance burden: external scripts and context files must stay in sync as thresholds, key names, and output formats evolve. That coordination challenge is the central topic of Part 6.

Recommendation: Start with JMCP Direct for development and integrate Redis early - key structure changes are cheap at this stage since data volume is low and there is nothing to migrate yet. Defer external Python until you have read Part 6. If you have the enhanced JMCP serveravailable, switch to artifact mode early - it gives you Redis storage and token isolation from day one without adding complexity. Migrate to Redis +

External Python when token usage or scale becomes an issue. The external Python analysis pattern - including a reference implementation - is covered in Part 6: Hybrid Architecture.

1.6 Installing Redis

Redis installation is straightforward on most platforms.

1.7 Ubuntu/Debian

```
sudo apt update
sudo apt install redis-server
# apt starts redis-server automatically after install
```

redis-cli is Redis's built-in command-line client - you use it throughout this blog to verify keys, check TTLs, and query stored data directly from the shell.

```
# Verify it is running
redis-cli ping
# Expected: PONG
```

1.8 macOS

```
brew install redis
# brew installs redis but does NOT start it - start it next
```

1.9 Starting Redis

1.9.1 Option A: Background Service (Recommended)

Survives terminal close, auto-restart on boot:

```
# macOS
brew services start redis

# Ubuntu/Debian
sudo systemctl start redis-server
sudo systemctl enable redis-server # Auto-start on boot
```

1.9.2 Option B: Foreground Process

For debugging, stops when terminal closes:

```
# macOS
/opt/homebrew/opt/redis/bin/redis-server /opt/homebrew/etc/redis.conf

# Ubuntu/Debian
redis-server /etc/redis/redis.conf
```

1.10 Verify Redis is Running

After starting Redis (macOS: any option above; Ubuntu/Debian: already running after install):

```
redis-cli ping
# Expected: PONG
# Error "Could not connect": Redis is not running - start it first
```

1.11 Context File Redis Prerequisites

When writing context-files or a skill, we instruct the LLM for an aliveness test first. You can embed Redis checks in your context files:

```
**Dependencies:**

- Redis server running on localhost:6379
- Python packages: redis, json
- Network access to routers via JMCP

**Redis Prerequisites:**

 Check if Redis is running:
```bash
redis-cli ping
# Expected: PONG (Redis is running)
# Error: "Could not connect" (Redis not running)
```

Start Redis if needed:
```bash
brew services start redis # macOS
sudo systemctl start redis-server # Ubuntu/Debian
```

Recommendation: Use service mode for normal operation.
```

1.12 Quick-Start: End-to-End Redis Example

Enhanced JMCP users: Everything in this section - writing data to Redis, maintaining the master index, setting TTLs, and reading data back - is handled automatically by the enhanced JMCP server in artifact mode. You never write these patterns yourself; you call **list_artifacts** / **read_artifact** instead. This section is for teams managing Redis storage directly (standard JMCP or any other collection path).

Before diving into the Redis concept in detail, here is the complete write -> read -> verify cycle for the ISIS LSDB use case. Every piece shown here is explained in depth in the sections that follow. The example below assumes Redis is running on the same host as the user's chat session. When Redis is running on a dedicated host, replace localhost as needed.

You may wonder why Python is needed. JMCP itself has no built-in capability to write to Redis. From a workflow perspective, the following happens:

- 1. The user invokes a JMCP tool such as execute_junos_command to run CLI commands on the routers.
- 2. The tool output arrives in the context window.
- 3. The LLM, guided by the context file, runs a Python snippet to save that output to Redis.
- 4. Once all commands have been collected, further Python scripts can retrieve the data from Redis for analysis - without reloading it into the context window.

The context file snippet below is what you save and load into your LLM session. The Python code is embedded directly inside it - the user never runs Python manually. The LLM reads the instructions, calls JMCP to collect data, and then executes the embedded snippets itself to write to and read from Redis.

Context file:

Note: The block below is a context file with embedded Python. Instructions such as “Execute the Python snippet below” are directed at the LLM - not the user. The LLM executes them autonomously as it follows the procedure.

This is the starting pattern. As the workflow scales (Scenario C and beyond), embedded Python is replaced by external scripts that the LLM invokes - rather than executes inline - keeping analysis logic out of the context window entirely. That evolution is covered in Part 6: Hybrid Architecture.

````
## Quick-Start: ISIS LSDB -> Redis

**Prerequisites:** See [Context File Redis Prerequisites](#context-file-redis-prerequisites) above.

-

### Phase 0: Collect and Store

**Procedure:**
1. Execute `show isis database | no-more` on R1 via JMCP (`execute_junos_command`)
2. Parse the output: extract total LSP count and the list of LSP IDs
3. Execute the Python snippet below, substituting the parsed values into `lsdb_data`:

```python
import redis, json, time

client = redis.Redis(host='localhost', port=6379, db=0)
ts = int(time.time())
router = "R1"

# Fill from parsed JMCP output (step 2 above).
# Example of what real parsed values look like:
# lsdb_data = {
# "lsp_count": 6,
# "lsps": ["vmx_nv7.00-00", "vmx_nv1.00-00", "vmx_nv2.00-00",
# "vmx_nv3.00-00", "vmx_nv4.00-00", "vmx_nv5.00-00"],
# }
lsdb_data = {"lsp_count": <N>, "lsps": ["<lsp-id>", ...]}

client.setex(
   name=f"isis:database:{router}:{ts}",
   time=604800, # 7 days - for TTL sizing rules see "Configuring N and TTL Together"
   value=json.dumps(lsdb_data),
)
client.hset("isis:routers:latest", router, ts)
# Register this run in the sorted-set run registry so Phase 3 can enumerate it
client.zadd("isis:phase0:runs", {str(ts): ts})
client.expire("isis:phase0:runs", 604800 * 2) # Keep run index longer than data
print(f"Stored: isis:database:{router}:{ts} (TTL 7 days)")
```

-
````

1.12.1 Phase 1: Read Back

Procedure: Execute the Python snippet below to confirm the stored data is retrievable:

```python
import redis, json

client = redis.Redis(host='localhost', port=6379, db=0)
router = "R1"

raw_ts = client.hget("isis:routers:latest", router)
if raw_ts is None:
   raise RuntimeError(f"No master index entry for {router} - run Phase 0 first")
ts = raw_ts.decode()
raw = client.get(f"isis:database:{router}:{ts}")
if raw is None:
   raise RuntimeError(f"Data for {router}:{ts} not found - TTL may have expired")
data = json.loads(raw)
print(f"Retrieved: {data['lsp_count']} LSPs for {router}")
```

CLI verification - run these after Phase 0 to confirm directly via redis-cli:

The key names used here (**isis:database:...**, **isis:routers:latest**, **isis:phase0:runs**) follow a naming convention explained in full in Key naming convention below.

```
# Confirm data is present
redis-cli GET isis:database:R1:$(redis-cli HGET isis:routers:latest R1)

# Check TTL is set (should be close to 604800 seconds)
redis-cli TTL isis:database:R1:$(redis-cli HGET isis:routers:latest R1)

# Confirm master index entry
redis-cli HGETALL isis:routers:latest

# Confirm run was registered in the run registry
redis-cli ZRANGE isis:phase0:runs 0 -1 WITHSCORES
# Expected: one entry, e.g.:
# 1) "1747123456"
# 2) "1747123456"
```

Intimidated by the embedded Python? You never need to write it yourself - the LLM generates it. Paste the prompt below into a new chat session and the LLM will produce a context file with the correct key structure, TTL, and Python snippets for your use case:

"*I want to collect CLI outputs from network routers via JMCP - for example show isis database - and store the parsed results in Redis on localhost. Once collection is complete, I want to read the data back from Redis for further analysis. Please generate a context file with embedded Python snippets covering both phases: the collection and store phase (Phase 0) and the read-back phase (Phase 1).*"

Embedded Python is a good quick-start pattern. For production workflows, the Python logic moves out of the context file and into an external script - the **Redis + External Python** pattern - where the script fetches data from Redis, analyses it entirely outside the LLM, and returns only a compact report (~2K regardless of fleet size). The LLM never loads the raw data at all. That transition is covered in the Performance Comparisons section below and in Part 6: Hybrid Architecture.

Once you have a Python script library alongside your context files, a new challenge appears: keeping the two in sync. If a threshold changes (e.g. staleness limit, TTL, look-back N), it must be updated in both the context file instructions and the Python script logic. The same applies to structural content: when a script gains a new output field, the context file must be updated to reference it correctly. Part 6 covers strategies for managing this drift - including how to use the LLM itself to detect and fix inconsistencies between your markdown and Python.

The sections below explain each part of this pattern in detail: TTL configuration and sizing, the Master Index HASH, Phase 3 trending with a sorted-set run registry, and when to replace this DIY plumbing with an external Python script.

1.13 Redis Storage: Master Index and TTL

These two mechanisms always work as a pair:

- **setex** writes a data key with automatic expiry (TTL)
- **hset** records the timestamp of that key in the Master Index (**isis:routers:latest** HASH)

The Master Index was introduced in Part 4: File Management where it solved a file-based storage problem: when JMCP collects data from multiple routers sequentially, each snapshot gets a slightly different timestamp. Without a central record of *which timestamp belongs to which router*, analysis phases have no way to detect

temporal skew - e.g. an LSDB sync check comparing R1's 10:00 AM snapshot against R2's 11:00 AM snapshot is comparing stale data and may silently produce incorrect results.

In Redis, the Master Index is a HASH (**isis:routers:latest**): one field per router, value is the latest collection timestamp. It serves the same consistency role as in Part 4, but now instead of pointing to files on disk it points to Redis keys. Without it, you would need to scan all keys (**KEYS isis:database:***) to find what's available - which is slow, unscoped, and error-prone at scale.

The **setex** + **hset** pair keeps the data key and the Master Index entry always in sync.

Key naming convention: consistent patterns enable efficient queries across all phases:

```
isis:database:{router}:{timestamp} # per-router data (with TTL)
isis:adjacency:{router}:{timestamp} # other command outputs follow same pattern
isis:routers:latest # Master Index HASH (no TTL)
baseline:isis_lsdb:{date} # baseline snapshots (with TTL)
trending:isis_lsdb:{date} # trending diff reports (with TTL)
```

Example with real values:

```
isis:database:R1:1733472000
isis:database:R2:1733472000
isis:routers:latest -> R1: 1733472000, R2: 1733472000
baseline:isis_lsdb:2025-02-21
```

Enhanced JMCP users: When using the enhanced JMCP server in artifact mode, you never write to Redis directly. The server maintains its own index structures automatically - a time-ordered sorted set (**jmcp:artifact:index:time**) as the run registry, and a per-artifact metadata HASH (**jmcp:artifact:meta:{run_id}**) that stores tool name, label, timestamps, and encoding. These serve the same consistency role as the manual **isis:routers:latest** HASH and run registry described below, but scoped to whole-run artifacts rather than per-router keys. Per-router staleness cannot occur because every artifact is a single barrier-synced batch snapshot. Access the index via **list_artifacts** / **read_artifact** - the run_id strings replace the manual key names shown below. The patterns here apply when you are managing Redis storage yourself (standard JMCP or any other collection path).

1.14 Writing Data: setex and hset Together

The key call is **setex** - it writes the value **and** sets the TTL atomically in one operation. The value= parameter must be a string, so the Python dict from parsing JMCP output is serialised first with **json.dumps()**:

Data transformation (logical flow - not executable code):

```
JMCP output (raw CLI text)
   -> LLM parses it into a Python dict e.g. {"lsp_count": 6, "lsps": [...]}
   -> json.dumps() serialises the dict to a JSON string
   -> setex() writes that string to Redis with the TTL
```

Every **setex** call is paired with an **hset** to update the Master Index:

```
import redis, json, time

client = redis.Redis(host='localhost', port=6379, db=0)
ts = int(time.time())

for router in routers:
   # `router_data` is a Python dict built from parsed JMCP output
   client.setex(
      name=f"isis:database:{router}:{ts}",
      time=604800, # TTL: 7 days in seconds (7 x 24 x 60 x 60)
      value=json.dumps(router_data)
   )
```

```
    # Update Master Index: track latest timestamp per router
    client.hset("isis:routers:latest", router, ts)

# Master Index itself carries no TTL - individual data keys expire after 7 days
```

Note: This snippet illustrates the **setex** + **hset** pair only. A complete Phase 0 write also registers the run in the sorted-set run registry via **client.zadd("isis:phase0:runs", ...)** - see Configuring N and TTL Together for the full write including the run registry.

1.15 Why TTL Matters

Without TTL: keys accumulate indefinitely, memory grows, stale data never clears, manual cleanup required.

With TTL: automatic expiration (e.g., 7 days), memory freed automatically, zero maintenance.

1.16 Querying: Reading from the Master Index

```
# Get all routers and their latest timestamps
master_index = client.hgetall("isis:routers:latest")

# Get data for a specific router (latest timestamp)
router = "R1"
ts = client.hget("isis:routers:latest", router).decode()
data = json.loads(client.get(f"isis:database:{router}:{ts}"))
```

Don't let the Python snippets, TTL values, and Master Index pattern intimidate you - the LLM generates all of this. Describe your goal in plain English (e.g., "*Read the latest ISIS LSDB data from Redis, verify freshness via the master index, and flag any stale entries*"), and the LLM will produce a context file with the correct key names, TTL configuration, and verification logic. You write the intent; the LLM writes the code.

One limitation of the embedded-Python approach shown here is worth keeping in mind: all data processing still happens inside the context window, which carries a real risk of token exhaustion at scale. Comparing large datasets - particularly in scheduled, periodic workflows - should be delegated to an external Python script that fetches and analyses data entirely outside the context window. For ad-hoc investigations on small fleets, inline processing works fine. Part 6 introduces the hybrid architecture that resolves this: an LLM that coordinates external Python scripts rather than running analysis itself.

1.17 Verifying TTL in shell

After storing, confirm the TTL is set correctly.

Step 1 - Find the timestamp (from the Master Index):

```
# One router
redis-cli HGET isis:routers:latest R1
# Example output: 1733472000

# All routers at once
redis-cli HGETALL isis:routers:latest
```

Step 2 - Check the TTL using that timestamp:

```
redis-cli TTL isis:database:R1:1733472000
# Output: 604800 (just stored) or lower number (aging)
# Output: -1 (no expiry set - TTL was not applied)
# Output: -2 (key doesn't exist)
```

Or combine both steps into a one-liner:

```
redis-cli TTL isis:database:R1:$(redis-cli HGET isis:routers:latest R1)
```

1.18 Context File Master Index Usage

The example below is a context file snippet for Phase 1 LSDB analysis backed by Redis. It illustrates how the Master Index and TTL patterns from the previous sections translate into a real, resilient workflow.

The **primary** / **fallback** structure is the key design decision: always read from the Master Index first and only issue a live JMCP query when the cached data is absent or stale. This matters for two reasons. First, it avoids re-querying routers that already have fresh data - keeping latency low and the collection snapshot temporally tight. Second, it makes the auto-recovery path explicit: when a router's data is stale or missing, the same embedded Python snippet from Phase 0 is reused to refresh it and update the Master Index, so the analysis step always sees a consistent, validated dataset.

The three-step structure is deliberate: step 1 is a single self-contained Python snippet that handles connection setup, freshness classification, and data retrieval in one go - producing a **fresh_data** dict and a **stale_or_missing** list; step 2 is a separate self-contained write snippet that the LLM executes once per stale or missing router; step 3 is intentionally open-ended - it delegates the LSDB sync check to the LLM's own reasoning rather than encoding rigid rules. Earlier parts of this series showed how to sharpen such open-ended steps by adding explicit thresholds and named checks. Apply the same technique here to harden this workflow for production use.

Design rule - no invented glue code: A well-formed context file leaves nothing for the LLM to invent *in the mechanical parts*. If the LLM synthesises new Python to bridge two steps - adding undefined variables, helper functions, or scoping logic absent from every snippet - the embedded code is incomplete. Treat synthesised glue as a diagnostic signal: find the missing piece and add it explicitly to the context file. The LLM's role is to execute the Python you wrote, not to complete it.

The analytical step is different - but it is not fully free-form either. The operator should define the specific checks, thresholds, and expected outcomes that matter for their environment: which LSP sets to compare, what counts as an asymmetry, what the acceptable L1/L2 count range is, and what output format the report should use. Encoding these explicitly ensures consistency across runs and makes failures reproducible. A well-written Step 3 looks like: "*Compare **lsps_l1** and **lsps_l2** sets across all routers in fresh_data. Flag any router whose L1 set differs from its L2 set. Flag any pair of routers with different L1 sets. Report: PASS if all sets are identical; FAIL with per-router diff otherwise.*" That is operator-defined reasoning - not invented glue code.

What belongs at the end of Step 3, after all the defined checks, is a short open-ended invitation: "*After completing the checks above, flag any anomaly or cross-domain signal you notice that the operator may not have anticipated.*" This gives the LLM permission to surface a correlation the operator did not think to ask for - a router whose LSP lifetime is consistently low across both levels, an asymmetry that points to a one-way adjacency, a pattern that only emerges when the full dataset is seen at once. The context file defines the floor; the open-ended sentence raises the ceiling. Both are intentional.

VS Code approval dialog - what it means and what it does not: When the Copilot agent runs a terminal command (including embedded Python), VS Code shows an approval prompt. This is a safety gate that asks "should the agent be allowed to run terminal commands at all?" - it is not a quality gate that

checks whether the code is pre-written or synthesised on the fly. VS Code has no visibility into whether the Python came from your context file or was invented by the LLM; both trigger the same prompt. The design rule above is the quality gate; VS Code cannot enforce it. To suppress the prompt, either click the dropdown arrow on **Allow -> Always Allow** (scoped to that command pattern, persists across sessions), or set "**github.copilot.chat.agent.autoRunCommand": true** in your VS Code **settings.json** to auto-approve all agent terminal commands globally.

[!] **Warning: autoRunCommand: true** removes the approval gate for *all* agent-initiated terminal commands in every workspace, not just the ones from your context file. A poorly written or adversarially crafted context file could execute destructive commands - file deletions, network changes, data overwrites - without any prompt. Prefer Always Allow (per command pattern) for targeted suppression; reserve **autoRunCommand** only for isolated lab environments where you control all context files loaded into the session.

The prompt below is ready to copy and paste directly into a new chat session. To try it: pick any 2 routers reachable via JMCP, paste the prompt, and let the agent run all three steps end to end.

````
## Phase 1: LSDB Analysis (Redis-backed)

**Data Source:**
- **Primary:** Redis master index `isis:routers:latest` (HASH)
- **Fallback:** If master index missing or data stale, re-query via JMCP

**Procedure:**
1. **Get Timestamps, Check Freshness, and Fetch Fresh Data:**
   Execute the snippet below. It connects to Redis, reads the master index, classifies each
       router as FRESH or STALE/MISSING, and loads all fresh data into `fresh_data`:

   ```python
   import redis, json, time

   client = redis.Redis(host='localhost', port=6379, db=0)
   now = int(time.time())
   STALE_THRESHOLD = 300 # seconds (5 minutes)
   TARGET_ROUTERS = None # None = all routers; set to e.g. ["vmx_nv3", "vmx_nv7"] to scope

   master_index = client.hgetall("isis:routers:latest")
   # Returns: {b'R1': b'1733472000', b'R2': b'1733472000', ...}

   fresh_data = {}
   stale_or_missing = [] # routers that need auto-recovery in step 2

   for router_b, ts_b in master_index.items():
      router = router_b.decode()
      if TARGET_ROUTERS and router not in TARGET_ROUTERS:
         continue
      ts = ts_b.decode()
      age = now - int(ts)
      if age < STALE_THRESHOLD:
         key = f"isis:database:{router}:{ts}"
         data = client.get(key)
         if data:
            fresh_data[router] = json.loads(data)
            print(f" FRESH: {router} (age={age}s)")
         else:
            print(f" MISSING key: {router}:{ts} (TTL may have expired)")
            stale_or_missing.append(router)
      else:
         print(f" STALE: {router} (age={age}s)")
         stale_or_missing.append(router)

   # Routers in TARGET_ROUTERS but absent from master index are also missing
   if TARGET_ROUTERS:
      for r in TARGET_ROUTERS:
         if r not in fresh_data and r not in stale_or_missing:
            print(f" MISSING (not in master index): {r}")
            stale_or_missing.append(r)

   print(f"\nFresh: {list(fresh_data.keys())}")
   print(f"Need recovery: {stale_or_missing}")
   ```
````

```
   > **Note:** This per-router freshness check is needed only when Redis is populated
       incrementally (one router at a time, e.g. sequential standard JMCP calls). When using the
        enhanced JMCP in artifact mode, every run is a single atomic barrier-synced snapshot -
       all routers share the same collection timestamp, so per-router staleness cannot occur.
       Skip steps 1--2 in that case; instead use `list_artifacts` to select a run whole.

2. **Auto-Recover Stale/Missing Data:**
   Execute JMCP queries for all STALE or MISSING routers to get fresh CLI output. Parse each
       result into a dict, then run the snippet below to write all recovered data to Redis in
       one pass and update the master index:

   ```python
   import redis, json, time

   client = redis.Redis(host='localhost', port=6379, db=0)
   ts_new = int(time.time())

   # `recoveries` = dict built from JMCP output for each stale/missing router:
   # { router_name: lsdb_dict, ... }
   # e.g.:
   # recoveries = {
   # "vmx_nv3": {"lsp_count_l1": 5, "lsp_count_l2": 5,
   # "lsps_l1": ["vmx_nv3.00-00", ...], "lsps_l2": [...]},
   # "vmx_nv7": {"lsp_count_l1": 5, "lsp_count_l2": 5,
   # "lsps_l1": ["vmx_nv3.00-00", ...], "lsps_l2": [...]},
   # }
   for router, lsdb_data in recoveries.items():
      client.setex(
         name=f"isis:database:{router}:{ts_new}",
         time=604800, # 7 days - must be >= MIN_TTL_SECONDS (see "Configuring N and TTL Together
             ")
         value=json.dumps(lsdb_data),
      )
      client.hset("isis:routers:latest", router, ts_new)
      print(f"Recovered: isis:database:{router}:{ts_new} (TTL 7 days)")

   # Register this recovery as a Phase 0 run so Phase 3 can include it
   client.zadd("isis:phase0:runs", {str(ts_new): ts_new})
   client.expire("isis:phase0:runs", 604800 * 2) # Keep run index longer than data
   print(f"Registered recovery run {ts_new} in isis:phase0:runs")
   ```

3. **Analyze Combined Dataset:**
   - Perform LSDB sync check on fresh + recovered data
   - Generate report
```

Want to generate above inline Python Redis-backed Phase 1 analysis snippet for your own workflow? **Ask your LLM:

> Generate a context file snippet for Phase 1 LSDB analysis (ISIS LSDB, standard JMCP). Requirements: 1. Step 1 - a single self-contained Python snippet that connects to Redis on localhost:6379, reads the master index **isis:routers:latest**, includes a **TARGET_ROUTERS = None** config variable (set to a list to scope to specific routers; None = all routers), classifies each router as FRESH (age < 5 minutes) or STALE/MISSING, loads all fresh data into a **fresh_data** dict, and collects stale/missing router names into a **stale_or_missing** list. 2. Step 2 - a separate self-contained Python snippet that iterates over a **recoveries** dict (built from JMCP output for all stale/missing routers in one pass) and writes each entry to Redis using setex (TTL 604800 s / 7 days) and updates the master index with **hset**. Include a commented example of the expected **recoveries** structure. 3. Step 3 - open-ended LSDB sync check across the combined dataset. The parsed JMCP data structure must separate L1 and L2 fields: **{lsp_count_l1, lsp_count_l2, lsps_l1, lsps_l2}**. No variables may be undefined between steps - each snippet must be fully self-contained with no glue code left for the LLM to invent.

Expect iteration. The prompt above went through several refinements before it produced a context file that ran without modification. Early versions omitted the **TARGET_ROUTERS** scoping variable and used a single-router write snippet rather than the **recoveries** batch dict - gaps that only became visible when the prompt was actually executed. This is normal: prompt intent is usually correct on the first attempt; completeness is

not. When a generated context file is missing inline Python for a specific case, fix it with a targeted follow-up: "*The generated context file is missing inline Python to handle . Extend it with a self-contained snippet that covers this, using the same key structure and variable names already present.*" Iterate in small, focused steps rather than rewriting the whole prompt from scratch.

1.19 Redis TTL and Phase 3 Trending

Trending requires at least two temporally separate snapshots. Persistent storage is therefore essential: without it there is nothing to compare against. The TTL configured in Redis directly controls how far back Phase 3 can look - once a key expires, that snapshot is gone. For long-lived comparisons (weeks or months), export snapshots to files or a non-memory-based database rather than relying on Redis alone.

Part 4 introduced Phase 3 trend analysis and stated that Redis TTL-based expiry of old runs and look-back window configuration are covered here.

1.20 TTL as the Look-Back Mechanism

When Phase 0 data is stored in Redis, TTL controls how far back Phase 3 (Trending) can look. The relationship is direct: set TTL >= the total span of the look-back window.

Look-back window (N runs)	Collection Interval	Min TTL needed	Recommended TTL
5 runs	every 6 h	30 h (108000 s)	7 days (604800 s)
5 runs	every 24 h	5 days (432000 s)	14 days (1209600 s)
10 runs	every 6 h	60 h (216000 s)	7 days (604800 s)

Rule: **min_TTL = N x collection_interval**. A 3x safety buffer protects against delayed Phase 3 runs or collection failures. A 7-day TTL (604800 s) works well as a default for most daily or hourly schedules.

1.21 Configuring N and TTL Together

MIN_TTL_SECONDS is the mathematical floor - the absolute minimum TTL for Phase 3 to reach back N runs at the configured collection interval. **TTL_SECONDS** is intentionally set above that floor: the default 604800 s (7 days) gives a 5.6x safety margin to absorb delayed Phase 3 runs, collection failures, and the need to retain data for debugging. The assert is a hard stop, not an invitation to set both values equal - if you change **LOOKBACK_N** or **COLLECTION_INTERVAL_H** and forget to adjust **TTL_SECONDS**, Phase 0 fails immediately with a clear message rather than silently dropping snapshots mid-window. Use the Contract Lint prompt to catch the same mismatch across your full context file.

N (look-back window size) and TTL are coupled - change one and verify the other is still sufficient:

```
# Phase 0 storage - set TTL to cover the look-back window
LOOKBACK_N = 5 # Number of Phase 0 runs Phase 3 will compare
COLLECTION_INTERVAL_H = 6 # Hours between Phase 0 runs

# INVARIANT: TTL_SECONDS >= LOOKBACK_N x COLLECTION_INTERVAL_H x 3600
# Why: Phase 3 reaches back N runs. The oldest run is (N-1) collection intervals
```

```
# old at best, but Phase 3 itself may run late - the formula is the hard floor.
# Example with defaults: 5 x 6 h x 3600 = 108 000 s minimum.
# 604 800 s (7 days) gives a 5.6x safety margin above that floor.
# If you change either parameter, recompute MIN_TTL_SECONDS and raise TTL_SECONDS.
# Linter: run the Contract Lint prompt (see "TTL alignment check" callout below)
# to have the LLM verify this invariant against your context file automatically.
MIN_TTL_SECONDS = LOOKBACK_N * COLLECTION_INTERVAL_H * 3600
TTL_SECONDS = 604800 # 7 days - safe default for N<=5, interval<=24h
assert TTL_SECONDS >= MIN_TTL_SECONDS, (
   f"TTL too short: {TTL_SECONDS}s < {MIN_TTL_SECONDS}s required "
   f"(LOOKBACK_N={LOOKBACK_N}, COLLECTION_INTERVAL_H={COLLECTION_INTERVAL_H}h)"
)

for router in routers:
   client.setex(
      name=f"isis:database:{router}:{ts}",
      time=TTL_SECONDS,
      value=json.dumps(data)
   )
   # Keep Master Index in sync - always pair with setex
   client.hset("isis:routers:latest", router, ts)

# Register this Phase 0 run in a sorted set so Phase 3 can enumerate all runs
# (score = unix timestamp for chronological ordering)
client.zadd("isis:phase0:runs", {str(ts): ts})
client.expire("isis:phase0:runs", TTL_SECONDS * 2) # Keep run index longer than data
```

Verify the run was registered - run this immediately after Phase 0 to confirm the sorted set was written:

```
redis-cli ZRANGE isis:phase0:runs 0 -1 WITHSCORES
# Expected: one entry per Phase 0 run, e.g.:
# 1) "1747123456"
# 2) "1747123456"
```

redis-cli ZRANGE .. Empty array? The sorted set is only written by the **client.zadd(...)** call above. If you ran earlier quick-start snippets that predate this snippet, those runs were never registered - the data keys exist in Redis but the run registry is empty. Re-run Phase 0 (or the recovery snippet in Step 2 of Phase 1) once; the set will populate immediately.

TTL alignment check: As a general practice, close every context-file working session with a linter prompt - see Part 3: Contract Lint. The **# INVARIANT:** keyword above is what makes the TTL constraint visible to the linter: when the linter reads the file it notices the **# INVARIANT:** marker, understands the relationship between **LOOKBACK_N**, **COLLECTION_INTERVAL_H**, and **TTL_SECONDS**, and explicitly checks whether the configured value sits above the floor. Without the **# INVARIANT:** comment, the linter sees three independent numeric variables and has no signal that they are coupled - the mismatch silently passes.

```
Read this context file as a linter. Flag inconsistencies, undefined or misdefined items.
```

1.22 Tracking Phase 0 Runs for Phase 3

Enhanced JMCP users: The server maintains its own artifact registry automatically. Use **list_artifacts(artifact_backend: redis, limit: N)** to enumerate all available runs - no sorted set required. The **zadd** pattern below is for self-managed Redis deployments only.

The Master Index (**isis:routers:latest**) tracks the latest timestamp per router. Phase 3 needs to enumerate all recent Phase 0 run timestamps. When managing Redis yourself, use a sorted set as the run registry. To get the N most recent runs for Phase 3:

```
# Get the N most recent runs (latest first)
# Upper index = LOOKBACK_N - 1 (e.g. 4 for LOOKBACK_N=5)
redis-cli ZREVRANGE isis:phase0:runs 0 4 WITHSCORES
```

Don't remember the exact syntax? Ask the LLM: "*What is the redis-cli command to list all available Phase 0 run timestamps from a sorted set?*" - it will produce the **ZRANGE** / **ZREVRANGE** commands above.

1.23 Handling Expired Runs in Phase 3

Phase 3 is fundamentally a diff operation - it compares the current snapshot against one or more earlier ones. A minimum of two valid runs in the look-back window is therefore required; with only one, there is nothing to compare against and Phase 3 must abort rather than silently produce a "no change" result.

If TTL is configured too short and Phase 0 data expires before Phase 3 runs, some timestamps in the sorted set will reference keys that no longer exist in Redis. The correct response is to skip those expired runs and check whether enough valid ones remain:

```
**Phase 3 Expiry Handling:**
1. Fetch N most recent run timestamps from `isis:phase0:runs` (sorted set)
2. For each timestamp {ts}, probe per router from the master index: `EXISTS isis:database:{
    router}:{ts}` > 0
3. If key returns nil (TTL expired):
   - Log: `WARN: Run {ts} - data expired, excluded from trend window`
   - Continue with remaining valid runs
4. If fewer than 2 valid runs remain:
   - Abort Phase 3: `INSUFFICIENT DATA - need >= 2 runs, found {n}`
   - Root cause: TTL too short or Phase 3 ran later than expected
   - Action: increase TTL, reduce N, or wait for more runs to accumulate
```

This is also why the **# INVARIANT** comment in "Configuring N and TTL Together" matters: if the invariant is violated, expiry handling is the first symptom - runs silently drop out of the window until Phase 3 can no longer form a valid comparison pair.

Want to generate this Phase 3 expiry-handling snippet yourself? Paste this into your LLM:

> *Generate a context file snippet for Phase 3 trending that compares the last two runs of **show isis database** stored in Redis. Include expiry handling: if a run's data key no longer exists (TTL expired), skip it and try the next older run. Abort with a clear error message if fewer than 2 valid, disjoint snapshots remain - never silently produce a 'no change' result from a single run.*

1.24 Summary

In this part, we covered:

- Redis installation and configuration
- TTL (Time To Live) for automatic data expiration
- Redis TTL as the Phase 3 look-back window: N-run configuration, run registry (sorted set), and expired-run handling
- Master Index pattern with Redis HASH
- Performance comparisons: JMCP Direct vs Redis + Inline Python vs Redis + External Python
- Redis storage patterns and key naming conventions
- When to use Redis vs Files (and hybrid approaches)

In Part 6, we'll explore hybrid LLM+Python architectures: Master Runbook patterns, keeping markdown and Python in sync, variable management, and when to use LLM analysis vs external scripts.

1.25 Next in the Series

Coming up in Part 6:

- Hybrid solution architecture (LLM + Python)
- Master Runbook as entry point
- Purpose-built logic files (analysis, collection, trending)
- Keeping markdown and Python scripts in sync - thresholds (TTL, staleness limits, look-back N) and structural content (output fields, key names)
- Variable management in hybrid approaches
- Choosing between LLM analysis and external scripts
- Scaling Redis for large fleets: connection pooling, key sharding, and Redis Cluster configuration

1.26 Glossary

- Artifact mode: An operating mode of the enhanced JMCP server where Phase 0 output is written directly to Redis or files, bypassing the LLM context window entirely. Data is then accessed via list_artifacts / read_artifact rather than being returned inline to the LLM.
- Background service: A process that runs independently of terminal sessions, typically configured to auto-start on system boot (e.g., via systemd or brew services).
- Baseline snapshot: A point-in-time capture of network state (e.g., ISIS LSDB, adjacency table) stored in Redis or files and used as a reference for detecting changes in subsequent Phase 3 trending comparisons.
- Context window: The finite memory space available to an LLM within a single session. All inputs, outputs, and conversation history consume tokens from this window; once full, earlier content is evicted.
- External Python script: An analysis script that runs as a standalone process outside the LLM session. It fetches data from Redis or files directly, performs computations in memory, and returns only a compact result report to the LLM - keeping token cost fixed regardless of fleet size.
- Foreground process: A process that runs attached to the current terminal session and stops when the terminal closes, useful for debugging.
- HASH: Redis data structure that stores field-value pairs, ideal for representing objects or mappings (e.g., master index tracking router timestamps).
- In-memory storage: Data storage in RAM rather than disk, providing sub-millisecond access times but requiring sufficient memory capacity.
- JMCP (Junos MCP server): A Model Context Protocol server that exposes Junos CLI commands as LLM-callable tools. Available as a standard version (returns raw CLI output to the client) and an enhanced version with built-in Redis/file storage that persists data directly without injecting it into the LLM context window.
- Key-value store: Database architecture where data is stored and retrieved using unique keys, enabling fast lookups by key name.
- Look-back window: The number of historical Phase 0 runs that Phase 3 trending analysis considers when detecting changes or drift. Controlled by the parameter N; requires TTL >= N x collection interval to ensure data is available.
- Master Index: A Redis HASH (key isis:routers:latest) that maps each router name to the timestamp of

its most recent Phase 0 data entry. Provides O(1) lookup of the latest snapshot for any router without scanning all keys. The index itself carries no TTL; individual data keys expire normally.

- Phase 0: The data-collection phase of the workflow. JMCP is used to query devices, outputs are parsed, and results are stored in Redis or files with a TTL. Phase 0 also registers the run timestamp in the sorted-set run registry.
- Phase 1: The analysis phase that operates on Phase 0 data. Reads the Master Index to locate the latest snapshot per router, checks freshness, triggers auto-recovery for stale or missing entries, and performs the primary analysis (e.g., LSDB sync check).
- Phase 2: The correlation phase that loads all Phase 1 domain reports, identifies cross-domain causal chains per router (e.g., interface errors -> adjacency flaps -> stale LSP), and writes an executive summary to results/phase2/. Phase 3 reads these summaries alongside the Phase 0 snapshots when trending across runs.
- Phase 3: The trending phase that compares N recent Phase 0 snapshots and their paired Phase 2 executive summaries to detect changes, recurring failures, and drift over time. Relies on TTL being set large enough to cover the full look-back window and on the run registry to enumerate available run timestamps.
- Redis: An in-memory data structure store providing exceptional performance (sub-millisecond access), TTL support for automatic expiration, and structured storage patterns for network data.
- Run registry: A Redis sorted set (e.g., isis:phase0:runs) that records the Unix timestamp of every completed Phase 0 run, scored by that timestamp for chronological ordering. Phase 3 queries this set to enumerate all available runs within the look-back window.
- Sorted set: Redis data structure that stores unique string members each associated with a floating-point score, ordered by score. Used as the Phase 0 run registry because it supports efficient range queries (e.g., “get the N most recent run timestamps”) via ZRANGE / ZREVRANGE.
- Stale data: Cached data that is too old to be reliable for current analysis, typically determined by comparing collection timestamp with current time.
- Token cost: The number of tokens consumed from the LLM context window by a given operation. Loading raw per-router data inline (Scenario B) causes token cost to scale with fleet size; routing analysis through an external Python script (Scenario C) keeps token cost fixed at approximately 2K regardless of fleet size.
- TTL (Time To Live): Automatic expiration mechanism where Redis keys are deleted after a specified duration (e.g., 7 days = 604800 seconds), eliminating manual cleanup requirements.
- Working data: Current operational data actively used for analysis, as opposed to permanent audit records or historical archives.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



